

DSA Week 2 Preparation Guide

Linked Lists, Two Pointers, Sliding Window, Binary Search, BFS/DFS, and More

Prepared for: Sriram | Date: May 4, 2026 | Estimated Study Time: 12-15 hours

Welcome to Week 2! If you've completed Week 1, you already have a solid foundation in arrays, hashing, and basic string manipulation. This week, we level up. You'll encounter new data structures like linked lists, powerful techniques like sliding window and binary search, and graph traversal strategies that unlock an entirely new class of problems. These are the patterns that interviewers love — and the ones that separate prepared candidates from the rest.

Don't be intimidated by the jump in difficulty. Every hard problem is built from simple ideas combined together. Take each problem one at a time, understand the *why* behind each approach, and you'll be amazed at how quickly these patterns click. Let's dive in.

Learning Objectives

By the end of this week, you will be able to:

- Reverse and traverse linked lists using pointer manipulation
- Apply two-pointer and sliding window techniques to optimize brute-force solutions
- Adapt binary search for non-standard sorted arrays
- Use BFS/DFS to solve grid-based and graph problems
- Recognize which pattern to apply when you see a new problem

Week 2 Overview

Below is a summary of all 11 problems you'll tackle this week. They are organized roughly by category and increasing difficulty.

#	Problem Name	Difficulty	Category / Pattern	LeetCode #
1	Reverse a Linked List	Easy	Linked List	206
2	Detect Cycle in a Linked List	Easy	Linked List / Fast-Slow Pointers	141
3	Container With Most Water	Medium	Two Pointers	11
4	Find Minimum in Rotated Sorted Array	Medium	Binary Search	153
5	Longest Repeating Character Replacement	Medium	Sliding Window	424
6	Longest Substring Without Repeating Characters	Medium	Sliding Window	3
7	Number of Islands	Medium	BFS / DFS	200
8	Remove Nth Node From End of List	Medium	Linked List / Two Pointers	19
9	Palindromic Substrings	Medium	Expand Around	647

#	Problem Name	Difficulty	Category / Pattern	LeetCode #
			Center	
10	Pacific Atlantic Water Flow	Medium	BFS / DFS (Multi-source)	417
11	Minimum Window Substring	Hard	Sliding Window	76

Key Patterns for This Week

Before diving into the individual problems, let's understand the six core patterns you'll use repeatedly. Recognizing these patterns is the single most important skill for coding interviews — once you see the pattern, the solution often writes itself.

1. Linked List Manipulation

When to use it: Any time you're working with a linked list and need to rearrange, reverse, or detect structural properties (like cycles). The key tools are pointer reversal (swapping `next` references), dummy nodes (to simplify edge cases at the head), and the fast/slow pointer technique (to find midpoints or detect cycles).

Why it works: Linked lists don't support random access, so you must solve problems by carefully walking through nodes and rewiring connections. Mastering pointer manipulation here builds intuition that transfers to trees, graphs, and more complex data structures.

2. Two Pointers

When to use it: When you have a sorted array or a linear structure and need to find a pair, range, or boundary that satisfies a condition. The classic variant uses two pointers starting at opposite ends and converging toward each other based on comparisons.

Why it works: By moving pointers intelligently (instead of checking every pair), you reduce $O(n^2)$ brute force to $O(n)$. The key insight is that each pointer movement eliminates an entire set of possibilities, which is why it's so efficient.

3. Sliding Window

When to use it: Problems involving contiguous subarrays or substrings where you need to find the longest, shortest, or optimal window satisfying a condition. The window expands on the right and contracts on the left as needed.

Why it works: Instead of recalculating properties from scratch for every possible subarray ($O(n^2)$ or worse), the sliding window maintains a running state and incrementally updates it. This gives you $O(n)$ performance for problems that look quadratic at first glance.

4. Binary Search (Modified)

When to use it: Whenever you see a sorted or partially sorted structure and need to find an element, boundary, or minimum/maximum. Modified binary search adapts the classic algorithm for non-standard situations like rotated arrays or search spaces where the "array" is implicit.

Why it works: Even when the array isn't perfectly sorted, at least one half is always sorted after a rotation. By comparing the mid element with the

endpoints, you can determine which half to search, maintaining $O(\log n)$ performance.

5. BFS/DFS (Graph Traversal)

When to use it: Grid problems (islands, water flow), connected components, shortest paths, or anything where you need to explore all reachable nodes from a starting point. DFS uses a stack (or recursion) to go deep first; BFS uses a queue to explore level by level.

Why it works: Both BFS and DFS systematically visit every reachable cell or node exactly once. By marking visited nodes, you avoid infinite loops and ensure complete coverage. The choice between BFS and DFS depends on whether you need shortest path (BFS) or just reachability (either works).

6. Expand Around Center

When to use it: Palindrome problems — specifically, counting or finding palindromic substrings. Instead of checking every possible substring, you treat each character (and each gap between characters) as a potential palindrome center and expand outward.

Why it works: Every palindrome has a center (one character for odd length, two characters for even length). Expanding from the center outward is a natural way to discover all palindromes, and it runs in $O(n^2)$ time, which is much better than the $O(n^3)$ brute force.

Problem 1: Reverse a Linked List

Easy

LeetCode #206 | Pattern: Linked List — Iterative Pointer Reversal

Problem Statement

Given the head of a singly linked list, reverse the list and return the new head. Each node points to the next node in the list, and the last node points to `None`. After reversal, the last node becomes the first, and all arrows point in the opposite direction.

Example 1:

Input: 1 → 2 → 3 → 4 → 5

Output: 5 → 4 → 3 → 2 → 1

Example 2:

Input: 1 → 2

Output: 2 → 1

Intuition & Approach

Think of a chain of people each pointing to the person in front of them. To reverse the chain, you need each person to turn around and point to the person *behind* them instead. You do this one person at a time, walking along the chain.

The algorithm uses three pointers:

1. **prev** — the node behind the current one (starts as `None`)

2. `curr` — the node we're currently processing (starts at `head`)
3. `next_node` — a temporary save of the next node (so we don't lose it when we reverse the link)

At each step: save the next node, reverse the current node's pointer to point backward, then advance `prev` and `curr` one step forward. When `curr` becomes `None`, `prev` is the new head.

Common Mistake

Forgetting to save `curr.next` before overwriting it. Once you set `curr.next = prev`, the original next node is lost forever unless you saved it first.

Python Solution

```
def reverseList(head):    # Initialize prev to None – this will become the
                           tail (end) of the reversed list    prev = None    # Start with curr at the
                           head of the original list    curr = head    # Walk through every node in
                           the list, one at a time    while curr:    # SAVE the next node before
                           we overwrite the link    # Without this, we'd lose the rest of the list
                           next_node = curr.next    # REVERSE the pointer: instead of pointing
                           forward,    # make this node point backward to prev    curr.next =
                           prev    # ADVANCE prev to the current node (prev moves one step
                           forward)    prev = curr    # ADVANCE curr to the saved next node
                           (curr moves one step forward)    curr = next_node    # When the loop
                           ends, curr is None and prev is the new head    return prev
```

Recursive Alternative

You can also reverse a linked list recursively: recurse to the end, then rewire pointers as the call stack unwinds. The recursive approach has $O(n)$ space due to the call stack, while the iterative approach above uses $O(1)$ space. For interviews, the iterative solution is generally preferred.

Complexity Analysis

Metric	Value	Explanation
Time Complexity	$O(n)$	We visit each of the n nodes exactly once.
Space Complexity	$O(1)$	Only three pointer variables used, regardless of list size.

Key Takeaways

- Reversing a linked list is a foundational technique — it appears as a sub-step in many harder problems (e.g., reversing portions of a list, palindrome linked lists).
- The "save-before-overwrite" pattern (saving `next` before changing `curr.next`) is critical for any pointer manipulation problem.
- Practice drawing out the pointer changes on paper — visual tracing makes linked list problems much clearer.

Problem 2: Detect Cycle in a Linked List Easy

LeetCode #141 | Pattern: Floyd's Cycle Detection (Tortoise and Hare)

Problem Statement

Given the head of a linked list, determine if the list has a cycle in it. A cycle exists if some node's next pointer points back to a previous node, creating a loop. Return `True` if a cycle exists, `False` otherwise.

Example 1:

Input: `3 → 2 → 0 → -4 → (back to 2)`

Output: `True` — the tail connects back to the second node.

Example 2:

Input: `1 → 2 → None`

Output: `False` — no cycle, the list ends normally.

Intuition & Approach

Imagine two runners on a circular track: one runs at normal speed, the other runs twice as fast. If the track is a loop, the fast runner will eventually lap the slow runner and they'll meet. If there's no loop (the track has an end), the fast runner simply reaches the finish line.

This is **Floyd's Cycle Detection Algorithm**:

4. Create two pointers: `slow` (moves 1 step at a time) and `fast` (moves 2 steps at a time).
5. Move both pointers forward simultaneously.
6. If `fast` reaches `None` — there is no cycle.
7. If `slow` and `fast` ever point to the same node — there is a cycle.

Why does this work? In a cycle, the fast pointer closes the gap between itself and the slow pointer by 1 node per iteration. If the cycle has length c ,

they are guaranteed to meet within c iterations after both enter the cycle. This eliminates the need for a hash set, giving us $O(1)$ space.

Key Insight

The fast pointer reduces the distance to the slow pointer by exactly 1 each step. Think of it as the fast pointer "chasing" the slow pointer — in a circle, the chase always ends in a meeting.

Python Solution

```
def hasCycle(head):    # Initialize two pointers, both starting at the head
    slow = head        fast = head    # Move fast by 2 and slow by 1 each
    iteration          # We check fast AND fast.next because fast moves 2 steps    #
    If either is None, we've reached the end → no cycle    while fast and
    fast.next:        # Slow pointer moves one step forward        slow =
    slow.next         # Fast pointer moves two steps forward        fast =
    fast.next.next    # If they meet at the same node, a cycle exists!
    # This check is AFTER moving because they both start at head    if slow
    == fast:          return True    # Fast pointer reached the end of the
    list → no cycle   return False
```

Complexity Analysis

Metric	Value	Explanation
Time Complexity	$O(n)$	In the worst case, the fast pointer traverses the list once before meeting slow (or reaching the end).
Space Complexity	$O(1)$	Only two pointer variables used. No extra data structures needed.

Key Takeaways

- Floyd's algorithm is the gold standard for cycle detection — memorize this pattern. It's also used in LeetCode 142 (find the *start* of the cycle).
 - The fast/slow pointer technique extends beyond cycles: it's used to find the middle of a linked list, detect palindromes, and more.
 - Always check `fast` and `fast.next` in the while condition to avoid null pointer errors.
-

Problem 3: Container With Most Water Medium

LeetCode #11 | Pattern: Two Pointers (Converging)

Problem Statement

You are given an array `height` where each element represents the height of a vertical line drawn at that index. Find two lines that, together with the x-axis, form a container that holds the most water. Return the maximum amount of water the container can hold.

Example:

Input: `height = [1, 8, 6, 2, 5, 4, 8, 3, 7]`

Output: 49

Explanation: The lines at index 1 (height 8) and index 8 (height 7) form the container. $\text{Area} = \min(8, 7) \times (8 - 1) = 7 \times 7 = 49$.

Intuition & Approach

The brute force approach checks every pair of lines — that's $O(n^2)$. Can we do better?

Start with the widest possible container (left pointer at position 0, right pointer at the last position). The area is $\min(\text{height}[\text{left}], \text{height}[\text{right}]) \times (\text{right} - \text{left})$. Now, which pointer should we move?

Greedy reasoning: Moving a pointer inward always *decreases* the width. The only way to potentially *increase* the area is to find a taller line. Since the area is limited by the *shorter* line, moving the shorter line inward gives us a chance of finding something taller. Moving the taller line inward can never help — the height is still capped by the shorter one, and the width just got smaller.

8. Set `left = 0, right = len(height) - 1`.
9. Calculate area, update max if larger.
10. Move the pointer pointing to the shorter line inward.
11. Repeat until the pointers meet.

Why Move the Shorter Line?

The water level is determined by the shorter line. Moving the taller line can only decrease or maintain the height constraint, while the width definitely decreases — so the area can only decrease. Moving the shorter line is the only move that has a *chance* of improving the result.

Python Solution

```
def maxArea(height):    # Start with pointers at the widest possible
    left = 0            # Track the maximum
    right = len(height) - 1
```

```

area we've seen so far      max_area = 0      # Keep narrowing the container
until the pointers meet     while left < right:  # Calculate the area
of the current container    # Height is limited by the shorter of the
two lines                   # Width is the distance between the two pointers
width = right - left        h = min(height[left], height[right])
area = h * width            # Update our best answer if this container is
larger                     max_area = max(max_area, area)      # Move the pointer at
the SHORTER line inward    # This is the greedy choice – only a taller
line can improve the area  if height[left] < height[right]:
left += 1                  else:                      right -= 1      return max_area

```

Complexity Analysis

Metric	Value	Explanation
Time Complexity	O(n)	Each pointer moves at most n times total; every element is visited at most once.
Space Complexity	O(1)	Only a few variables used regardless of input size.

Key Takeaways

- This is a classic example of the **converging two-pointer** pattern. Whenever you see "find an optimal pair" in a linear structure, think two pointers.
 - The greedy reasoning — "move the shorter line because that's the bottleneck" — is a common interview talking point. Practice articulating *why* it's correct.
 - This problem connects to "Trapping Rain Water" (LeetCode 42), a harder variant with a similar two-pointer approach.
-

Problem 4: Find Minimum in Rotated Sorted Array Medium

LeetCode #153 | Pattern: Modified Binary Search

Problem Statement

An array sorted in ascending order has been rotated between 1 and n times. For example, `[0, 1, 2, 4, 5, 6, 7]` might become `[4, 5, 6, 7, 0, 1, 2]`. Find the minimum element in the rotated array. You may assume all elements are unique.

Example 1:

Input: `nums = [3, 4, 5, 1, 2]`

Output: `1`

Example 2:

Input: `nums = [4, 5, 6, 7, 0, 1, 2]`

Output: `0`

Intuition & Approach

In a normally sorted array, the minimum is simply the first element. But after rotation, the array is split into two sorted halves with a "pivot" point where the minimum lives. We need to find this pivot efficiently.

Key observation: In a rotated sorted array, at least one half (left or right of mid) is always properly sorted. We can use this to decide which half contains the minimum:

- 12.If `nums[mid] > nums[right]` — the rotation point (minimum) must be in the right half. Move `left = mid + 1`.
- 13.If `nums[mid] <= nums[right]` — the right half is properly sorted, so the minimum is at `mid` or to its left. Move `right = mid`.

⚠ Common Mistake

Don't compare `mid` with `left` — it creates ambiguity when `left == mid`. Always compare `nums[mid]` with `nums[right]` for clean, bug-free binary search in rotated arrays.

Python Solution

```
def findMin(nums):    # Standard binary search setup    left = 0    right = len(nums) - 1    # Binary search narrows down to the single minimum element    while left < right:    # Find the middle index        mid = (left + right) // 2        # CASE 1: mid element is greater than the rightmost element        # This means the "break point" (where rotation happened) is to the RIGHT        # So the minimum must be in the right half (excluding mid)        if nums[mid] > nums[right]:            left = mid + 1        # CASE 2: mid element is less than or equal to the rightmost element        # This means the right half is properly sorted        # The minimum could be at mid or to the LEFT of mid        else:            right = mid    # When left == right, we've found the minimum    return nums[left]
```

Complexity Analysis

Metric	Value	Explanation
Time Complexity	$O(\log n)$	We halve the search space each iteration, just like standard binary search.
Space Complexity	$O(1)$	Only a few pointer variables used.

Key Takeaways

- Modified binary search is a recurring interview pattern. The trick is always: "which half can I safely eliminate?"
 - Compare with `nums[right]`, not `nums[left]`, to avoid edge case bugs.
 - This problem is a stepping stone to "Search in Rotated Sorted Array" (LeetCode 33), which combines finding the pivot with searching for a target.
-

Problem 5: Longest Repeating Character Replacement Medium

LeetCode #424 | Pattern: Sliding Window

Problem Statement

Given a string `s` and an integer `k`, you can choose any character in the string and change it to any other uppercase English letter. You can do this at most `k` times. Return the length of the longest substring containing the same letter you can get after performing the operations.

Example 1:

Input: `s = "AABABBA"`, `k = 1`

Output: 4

Explanation: Replace the one 'B' in "AABA" to get "AAAA", which has length 4.

Example 2:

Input: `s = "ABAB"`, `k = 2`

Output: 4

Explanation: Replace both 'A's or both 'B's to get a uniform string of length 4.

Intuition & Approach

The key insight is this: for any window of characters, the number of characters you need to replace equals `window_length - count_of_most_frequent_char`. If this number is $\leq k$, the window is valid (you can make all characters the same with at most k replacements).

14. Maintain a sliding window with left and right pointers.
15. Track the frequency of each character in the current window.
16. Track `max_freq` — the count of the most frequent character in the window.
17. If `window_length - max_freq > k`, the window is invalid $\rightarrow$ shrink from the left.
18. Update the maximum valid window length.

Key Insight

You might wonder: when we shrink the window, should we update `max_freq` downward? Technically, `max_freq` might be stale (too high). But that's OK! A stale `max_freq` only makes the window condition stricter, never more lenient. The result is still correct, and we avoid an expensive "find the new max" step.

Python Solution

```
def characterReplacement(s, k):    # Dictionary to count frequency of each
character in current window    count = {}    # Left pointer of the sliding
window    left = 0    # Tracks the highest frequency of any single
character in the window    max_freq = 0    # Tracks the longest valid
window we've found    result = 0    # Right pointer expands the window one
character at a time    for right in range(len(s)):    # Add the new
character (at right pointer) to our frequency count    count[s[right]] =
count.get(s[right], 0) + 1    # Update max_freq if this character now
has the highest frequency    max_freq = max(max_freq, count[s[right]])
# Calculate the window length    window_length = right - left + 1
# If the number of characters to replace exceeds k, shrink the window
# Characters to replace = window_length - max_freq    if window_length -
max_freq > k:    # Remove the leftmost character from the frequency
count    count[s[left]] -= 1    # Shrink window from the
left    left += 1    # Update result with the current (valid)
window size    result = max(result, right - left + 1)    return result
```

Complexity Analysis

Metric	Value	Explanation
Time Complexity	O(n)	Each character is added and removed from the window at most once.
Space Complexity	O(1)	The frequency map has at most 26 entries (uppercase English letters).

Key Takeaways

- The " $\text{window_length} - \text{max_freq} \leq k$ " condition is the heart of this problem. Understanding this formula is more important than memorizing the code.
- Not updating `max_freq` when shrinking the window is a subtle but valid optimization — it avoids O(26) scans while still producing the correct answer.

- This problem teaches the variable-size sliding window pattern, which applies to many substring problems.
-

Problem 6: Longest Substring Without Repeating Characters

Medium

LeetCode #3 | Pattern: Sliding Window with HashSet

Problem Statement

Given a string `s`, find the length of the longest substring without repeating characters. A substring is a contiguous sequence of characters within the string.

Example 1:

Input: `s = "abcabcbb"`

Output: 3

Explanation: The answer is `"abc"`, with length 3.

Example 2:

Input: `s = "bbbbbb"`

Output: 1

Explanation: The answer is `"b"`, with length 1 (every character is the same).

Intuition & Approach

This is one of the most classic sliding window problems and an excellent first problem for learning the pattern. The idea is straightforward:

19. Maintain a window using `left` and `right` pointers.
20. Use a set to track which characters are currently in the window.
21. Expand `right`: if the new character is NOT in the set, add it and update the max length.
22. If the new character IS already in the set, shrink from the left by removing `s[left]` from the set and moving `left` forward, until the duplicate is removed.

This guarantees every window contains unique characters only. The result is the maximum window size encountered.

Common Mistake

Some beginners try to jump `left` directly to the position after the duplicate. While this works with a `HashMap` (storing character indices), it's trickier to implement correctly. The set-based approach with a while loop is safer and easier to debug.

Python Solution

```
def lengthOfLongestSubstring(s):    # A set to track characters currently
inside our window    char_set = set()    # Left pointer of the sliding
window    left = 0    # Track the longest valid substring length found
max_length = 0    # Right pointer expands the window one character at a
time    for right in range(len(s)):    # If the character at 'right' is
already in our window,    # we have a duplicate – shrink the window from
the left    # until the duplicate is removed    while s[right] in
char_set:    # Remove the leftmost character from the set
char_set.remove(s[left])    # Move the left pointer forward (shrink
the window)    left += 1    # Now s[right] is not in the set,
so add it    char_set.add(s[right])    # Update max_length with
```

```
the current window size          # Window size = right - left + 1
max_length = max(max_length, right - left + 1)    return max_length
```

Complexity Analysis

Metric	Value	Explanation
Time Complexity	$O(n)$	Each character is added to and removed from the set at most once. The inner while loop doesn't make it $O(n^2)$ — total removals across all iterations is at most n .
Space Complexity	$O(\min(n, m))$	Where m is the size of the character set (e.g., 26 for lowercase letters, 128 for ASCII). The set stores at most $\min(n, m)$ characters.

Key Takeaways

- This is the quintessential sliding window problem — if you master this one, the pattern transfers to dozens of similar problems.
 - The set-based approach is clean and intuitive. For an optimized version, use a HashMap storing the last index of each character to jump `left` directly.
 - Practice explaining why the time complexity is $O(n)$ despite the nested while loop — interviewers love this question.
-

Problem 7: Number of Islands

Medium

LeetCode #200 | Pattern: BFS / DFS — Graph Traversal

Problem Statement

Given a 2D grid of '1's (land) and '0's (water), count the number of islands. An island is surrounded by water and is formed by connecting adjacent lands horizontally or vertically. You may assume all four edges of the grid are surrounded by water.

Example:

```
Input:  [ ["1","1","0","0","0"], ["1","1","0","0","0"],  
         ["0","0","1","0","0"], ["0","0","0","1","1"]] Output: 3
```

Intuition & Approach

Think of the grid as a map. You're scanning it from top-left to bottom-right. Every time you step on a piece of land ('1') that you haven't visited before, you've discovered a new island. You then explore the entire island (using DFS or BFS) and mark every connected land cell as visited so you don't count it again.

23. Iterate through every cell in the grid.
24. When you find a '1', increment the island count.
25. Run DFS from that cell: visit all connected '1' cells and mark them as '0' (or visited) so they won't be counted again.
26. Continue scanning for the next unvisited '1'.

Key Insight

Marking cells as '0' (sinking the island) is a space-efficient alternative to maintaining a separate "visited" matrix. If you can't modify the input, use a visited set instead.

Python Solution

```
def numIslands(grid):
    # Edge case: empty grid
    if not grid:
        return 0
    # Get grid dimensions
    rows = len(grid)
    cols = len(grid[0])
    # Counter for the number of islands found
    island_count = 0

    def dfs(r, c):
        """Recursively visit all connected land cells and 'sink' them."""
        # BASE CASE: stop if out of bounds or if the cell is water ('0')
        if r < 0 or r >= rows or c < 0 or c >= cols or grid[r][c] == '0':
            return
        # Mark this cell as visited by setting it to '0' (sinking the land)
        grid[r][c] = '0'
        # Explore all 4 adjacent cells (up, down, left, right)
        dfs(r - 1, c) # Up
        dfs(r + 1, c) # Down
        dfs(r, c - 1) # Left
        dfs(r, c + 1) # Right

    # Scan every cell in the grid
    for r in range(rows):
        for c in range(cols):
            # When we find unvisited land, it's a NEW island
            if grid[r][c] == '1':
                island_count += 1
                dfs(r, c)
    # Sink the entire island so we don't count it again
    return island_count
```

Complexity Analysis

Metric	Value	Explanation
Time Complexity	$O(m \times n)$	Every cell is visited at most once by the main loop and at most once by DFS.
Space Complexity	$O(m \times n)$	Worst case for the DFS recursion stack (if the entire grid is land, the stack goes $m \times n$ deep).

Key Takeaways

- This is the gateway problem for grid-based graph traversal. Master this pattern, and problems like "Max Area of Island", "Surrounded Regions", and "Rotting Oranges" become much easier.
 - BFS works equally well here — use a queue instead of recursion. BFS avoids deep recursion stack issues for very large grids.
 - The "sink the island" technique (modifying the grid in-place) is a common trick to avoid extra space for a visited set.
-

Problem 8: Remove Nth Node From End of List Medium

LeetCode #19 | Pattern: Linked List / Two Pointers (Fast-Slow with Gap)

Problem Statement

Given the head of a linked list, remove the n -th node from the **end** of the list and return the modified list's head.

Example 1:

Input: $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$, $n = 2$

Output: $1 \rightarrow 2 \rightarrow 3 \rightarrow 5$

Explanation: The 2nd node from the end is 4, which is removed.

Example 2:

Input: 1, $n = 1$

Output: [] (empty list — the only node is removed).

Intuition & Approach

The challenge is finding the n -th node from the *end* in a single pass. The trick: use two pointers with a gap of exactly n nodes between them.

27. Create a **dummy node** that points to `head`. This handles the edge case where we need to remove the head itself.
28. Move the `fast` pointer $n + 1$ steps ahead (starting from dummy).
29. Move both `fast` and `slow` forward until `fast` reaches the end.
30. Now `slow` is right before the node to delete. Skip over it: `slow.next = slow.next.next`.

Why Use a Dummy Node?

Without a dummy node, removing the head requires special-case code. The dummy node creates a uniform structure so the same logic works whether we're removing the first, middle, or last node. This is a standard linked list technique — use it whenever deletion is involved.

Python Solution

```
def removeNthFromEnd(head, n):    # Create a dummy node pointing to head
# This simplifies edge cases (e.g., removing the head node)    dummy =
ListNode(0)    dummy.next = head    # Both pointers start at the dummy
node    fast = dummy    slow = dummy    # Move fast pointer (n + 1) steps
ahead    # This creates a gap of exactly n nodes between fast and slow
for i in range(n + 1):        fast = fast.next        # Move both pointers
forward until fast reaches the end (None)    # When fast is at None, slow is
right BEFORE the node to remove    while fast:        fast = fast.next
slow = slow.next    # Skip over the target node (delete it)    # slow.next
is the node we want to remove    slow.next = slow.next.next    # Return
dummy.next (not head, because head might have been removed!)    return
dummy.next
```

Complexity Analysis

Metric	Value	Explanation
Time Complexity	$O(n)$	We traverse the list once with the fast pointer and once (partially) with the slow pointer — still $O(n)$ total.
Space Complexity	$O(1)$	Only a few pointer variables and one dummy node.

Key Takeaways

- The "gap technique" with fast/slow pointers is elegant — it converts a two-pass problem (count length, then find position) into a single-pass solution.
- Always use a dummy node when a linked list problem involves deletion. It eliminates head-removal edge cases.
- Return `dummy.next`, not `head`, because the original head might have been the node you deleted.

Problem 9: Palindromic Substrings Medium

LeetCode #647 | Pattern: Expand Around Center

Problem Statement

Given a string `s`, return the number of palindromic substrings in it. A string is a palindrome when it reads the same backward as forward. A substring is a contiguous sequence of characters within the string.

Example 1:

Input: `s = "abc"`

Output: 3

Explanation: Three palindromic substrings: `"a"`, `"b"`, `"c"`.

Example 2:

Input: `s = "aaa"`

Output: 6

Explanation: `"a"`, `"a"`, `"a"`, `"aa"`, `"aa"`, `"aaa"`.

Intuition & Approach

Every palindrome has a center. For odd-length palindromes (like `"aba"`), the center is a single character. For even-length palindromes (like `"abba"`), the center is between two characters.

The **expand around center** technique:

31. For each index `i` in the string, try expanding outward in both directions.
32. Do this **twice** at each position: once for odd-length palindromes (center = `i`), and once for even-length palindromes (center = between `i` and `i+1`).
33. As long as the characters on both sides match, the expansion continues and each valid expansion counts as a palindrome.

Python Solution

```
def countSubstrings(s):    # Counter for total palindromic substrings
count = 0    def expand(left, right):    """Expand outward from center
and count palindromes found."""    nonlocal count    # Keep
expanding as long as:    # 1. We're within bounds    # 2.
Characters on both sides match    while left >= 0 and right < len(s) and
s[left] == s[right]:    # This is a valid palindrome – count it!
count += 1    # Expand outward: move left pointer left, right
pointer right    left -= 1    right += 1    # Try every
possible center position    for i in range(len(s)):    # ODD-length
palindromes: single character center    # e.g., "aba" has center at 'b'
expand(i, i)    # EVEN-length palindromes: two character center
# e.g., "abba" has center between the two 'b's    expand(i, i + 1)
return count
```

Complexity Analysis

Metric	Value	Explanation
Time Complexity	$O(n^2)$	For each of the n centers, expansion can go up to $n/2$ steps. Total: $O(n \times n/2) = O(n^2)$.
Space Complexity	$O(1)$	Only a counter and pointer variables used — no extra data structures.

Key Takeaways

- The expand-around-center technique is the go-to approach for palindrome problems. It's simpler and more space-efficient than dynamic programming for this particular problem.
- Always handle both odd and even length palindromes — forgetting even-length is a very common bug.
- This same technique is used in "Longest Palindromic Substring" (LeetCode 5), where instead of counting, you track the longest expansion.

Problem 10: Pacific Atlantic Water Flow Medium

LeetCode #417 | Pattern: BFS / DFS — Multi-source Traversal

Problem Statement

There is an $m \times n$ rectangular island that borders both the Pacific Ocean (top and left edges) and the Atlantic Ocean (bottom and right edges). The island receives rain, and water can flow to neighboring cells (up, down, left, right) if the neighboring cell's height is **less than or equal to** the current cell's height. Water can also flow into the ocean.

Return a list of grid coordinates where water can flow to **both** the Pacific and Atlantic oceans.

Example:

```
Input: heights = [[1,2,2,3,5],           [3,2,3,4,4],
                  [2,4,5,3,1],           [6,7,1,4,5],           [5,1,1,2,4]]
Output: [[0,4], [1,3], [1,4], [2,2], [3,0], [3,1], [4,0]]
```

Intuition & Approach

The naive approach — checking from every cell if water can flow to both oceans — is extremely expensive. The key insight is to **reverse the problem**: instead of flowing water *from* each cell *to* the ocean, start *from* the ocean and flow *inward* (uphill).

34. Start DFS/BFS from all cells bordering the Pacific Ocean (top row + left column). Mark all cells reachable from the Pacific.

35. Start DFS/BFS from all cells bordering the Atlantic Ocean (bottom row + right column). Mark all cells reachable from the Atlantic.

36. The answer is the **intersection** — cells reachable from both oceans.

When flowing "inward" (reverse direction), water flows to a neighbor only if the neighbor's height is **greater than or equal to** the current cell (going uphill, because in reality water flows downhill to reach the ocean).

Key Insight

Reversing the flow direction is a powerful technique. Instead of asking "from cell X, can water reach the ocean?" (hard), ask "from the ocean, which cells can be reached going uphill?" (much easier with multi-source BFS/DFS).

Python Solution

```
def pacificAtlantic(heights):    # Edge case: empty grid    if not heights:
    return []    rows = len(heights)    cols = len(heights[0])    # Sets to
    track which cells can reach each ocean    pacific_reachable = set()
    atlantic_reachable = set()    def dfs(r, c, reachable, prev_height):
    """DFS that flows UPHILL from the ocean inward."""    # Stop if: out of
    bounds, already visited, or can't flow uphill    if (r < 0 or r >= rows
    or c < 0 or c >= cols    or (r, c) in reachable
    or heights[r][c] < prev_height):    return    # Mark this cell
    as reachable from the current ocean    reachable.add((r, c))    #
    Explore all 4 neighbors, passing current cell's height as threshold
    dfs(r + 1, c, reachable, heights[r][c]) # Down    dfs(r - 1, c,
    reachable, heights[r][c]) # Up    dfs(r, c + 1, reachable, heights[r]
    [c]) # Right    dfs(r, c - 1, reachable, heights[r][c]) # Left    #
    Start DFS from PACIFIC borders (top row and left column)    for c in
    range(cols):    dfs(0, c, pacific_reachable, heights[0][c])    # Top
    row    for r in range(rows):    dfs(r, 0, pacific_reachable, heights[r]
    [0])    # Left column    # Start DFS from ATLANTIC borders (bottom row
    and right column)    for c in range(cols):    dfs(rows - 1, c,
    atlantic_reachable, heights[rows - 1][c]) # Bottom row    for r in
    range(rows):    dfs(r, cols - 1, atlantic_reachable, heights[r][cols -
    1]) # Right column    # The answer is the INTERSECTION of both sets    #
    Cells that can reach BOTH oceans    result = []    for r in range(rows):
    for c in range(cols):    if (r, c) in pacific_reachable and (r, c)
    in atlantic_reachable:    result.append([r, c])    return
    result
```

Complexity Analysis

Metric	Value	Explanation
Time Complexity	$O(m \times n)$	Each cell is visited at most twice (once per ocean's DFS). Total work is proportional to grid size.
Space Complexity	$O(m \times n)$	Two sets storing reachable cells, plus DFS recursion stack in the worst case.

Key Takeaways

- The "reverse the flow" insight transforms an expensive problem into an elegant one. Always ask yourself: "Can I approach this from the other direction?"
- Multi-source BFS/DFS — starting traversal from multiple points simultaneously — is a pattern used in "Rotting Oranges", "Walls and Gates", and similar problems.
- Using set intersection to combine results from independent traversals is a clean and efficient approach.

Problem 11: Minimum Window Substring Hard

Problem Statement

Given two strings `s` and `t`, return the minimum window substring of `s` such that every character in `t` (including duplicates) is included in the window. If no such window exists, return the empty string `""`.

Example 1:

Input: `s = "ADOBECODEBANC"`, `t = "ABC"`

Output: `"BANC"`

Explanation: The minimum window substring containing 'A', 'B', and 'C' is `"BANC"`.

Example 2:

Input: `s = "a"`, `t = "a"`

Output: `"a"`

Intuition & Approach

This is the hardest problem of the week and a classic advanced sliding window problem. The core idea: expand the right pointer until you have a valid window (contains all characters from `t`), then shrink the left pointer to find the smallest valid window.

The **"formed" counter trick** is the key optimization:

- Maintain a `HashMap need` with the count of each character required from `t`.
- Maintain a `HashMap window` with the count of each character in the current window.
- Track `have`: how many *unique characters* in the window have met their required count.

- Track `need_count`: how many unique characters we need in total (the size of `need`).
- When `have == need_count`, the window is valid → try shrinking from the left.

This avoids comparing the entire frequency maps at each step (which would be $O(52)$ or more), reducing the check to a simple $O(1)$ integer comparison.

The "Formed" Counter Trick

Instead of checking "does my window contain all required characters?" by comparing two full HashMaps (expensive), maintain a single integer `have`. Increment it when a character's window count reaches its required count; decrement it when it drops below. When `have == need_count`, the window is valid. This is the pattern for all "minimum window" problems.

Python Solution

```
def minWindow(s, t):    # Edge case: if t is empty, no window needed    if
not t:        return ""    # Build a frequency map of characters we NEED
from string t    need = {}    for char in t:        need[char] =
need.get(char, 0) + 1    # Frequency map of characters in our current
WINDOW    window = {}    # 'have' = number of unique chars in window that
meet the required count    # 'need_count' = number of unique chars we need
to satisfy    have = 0    need_count = len(need)    # Track the best
(smallest) valid window found so far    # result stores [left_index,
right_index], result_len stores its length    result = [-1, -1]
result_len = float("infinity")    # Left pointer of the sliding window
left = 0    # Right pointer expands the window    for right in
range(len(s)):        # Add the character at 'right' to our window frequency
map        char = s[right]        window[char] = window.get(char, 0) + 1
# Check if this character's count in the window now MEETS the requirement
# Only increment 'have' when we EXACTLY meet the requirement (not when we
exceed it)        if char in need and window[char] == need[char]:
have += 1        # While the window is valid (contains all required
characters),        # try to SHRINK it from the left to find a smaller valid
window        while have == need_count:            # Update the best result
if this window is smaller                window_size = right - left + 1
if window_size < result_len:                    result = [left, right]
result_len = window_size            # Remove the leftmost character from
the window            left_char = s[left]            window[left_char] -= 1
# If removing this character causes us to no longer meet its requirement,
```

```
# decrement 'have' — the window is no longer valid if left_char
in need and window[left_char] < need[left_char]:     have -= 1
# Shrink the window from the left left += 1 # Return the
smallest valid window, or "" if none was found l, r = result return
s[l:r + 1] if result_len != float("infinity") else ""
```

Complexity Analysis

Metric	Value	Explanation
Time Complexity	$O(n)$	Where $n = \text{len}(s)$. Both left and right pointers traverse s at most once. HashMap operations are $O(1)$.
Space Complexity	$O(m)$	Where m is the size of the character set. The two HashMaps store at most m entries each.

Key Takeaways

- This is the most advanced sliding window problem you'll encounter in interviews. The "formed" counter pattern (`have / need_count`) is essential — commit it to memory.
 - The two-phase approach — "expand right until valid, then shrink left until invalid" — is the blueprint for all minimum-window problems.
 - Don't be discouraged if this takes multiple attempts. Understanding this problem deeply means you can tackle almost any sliding window variant.
-

Weekly Summary & Study Strategy

Pattern Frequency This Week

Understanding which patterns appear most frequently helps you prioritize your study time.

Pattern	Problems	Count
Sliding Window	#5 Longest Repeating Character Replacement, #6 Longest Substring Without Repeating Characters, #11 Minimum Window Substring	3
Linked List Manipulation	#1 Reverse a Linked List, #2 Detect Cycle, #8 Remove Nth Node From End	3
BFS / DFS (Graph Traversal)	#7 Number of Islands, #10 Pacific Atlantic Water Flow	2
Two Pointers	#3 Container With Most Water, #8 Remove Nth Node From End	2
Binary Search	#4 Find Minimum in Rotated Sorted Array	1
Expand Around Center	#9 Palindromic Substrings	1

How to Recognize the Right Pattern

If You See...	Think...
"Linked list" + "reverse" or "cycle" or "nth from end"	Linked List pointer manipulation, fast/slow pointers
"Substring" or "subarray" + "longest/shortest" + some constraint	Sliding Window
"Sorted array" + "find/search" + $O(\log n)$ expected	Binary Search (possibly modified)
"Grid" + "connected" or "count regions" or "flow/fill"	BFS / DFS
"Two ends" + "maximize/minimize a pair" in linear structure	Two Pointers (converging)
"Palindrome" + "count" or "longest"	Expand Around Center

Suggested Study Order

If you have limited time, tackle the problems in this order to build skills progressively:

Day	Problems	Rationale
Day 1	#1 Reverse a Linked List, #2 Detect Cycle in a Linked List	Start with Easy problems to build confidence and learn linked list basics.
Day 2	#3 Container With Most Water, #8 Remove Nth Node From End	Practice two-pointer technique in two different contexts (array and linked list).
Day 3	#6 Longest Substring Without Repeating Characters, #5 Longest Repeating Character	Learn sliding window with the classic problem first (#6), then the trickier variant (#5).

Day	Problems	Rationale
	Replacement	
Day 4	#4 Find Minimum in Rotated Sorted Array, #9 Palindromic Substrings	Modified binary search and expand-around-center — two distinct techniques.
Day 5	#7 Number of Islands, #10 Pacific Atlantic Water Flow	Grid traversal day — start with the simpler DFS problem, then tackle multi-source traversal.
Day 6	#11 Minimum Window Substring	Dedicate a full session to the hardest problem. Review sliding window concepts from Day 3 first.
Day 7	Review all 11 problems	Re-solve any problems that gave you trouble. Focus on explaining the approach out loud.

Final Tips for Success

- **Understand before memorizing.** If you understand *why* an approach works, you can reconstruct the solution even if you forget the code. Focus on the intuition sections above.
- **Practice explaining out loud.** In interviews, communication matters as much as code. Narrate your thought process as you solve each problem.
- **Draw it out.** For linked lists and grids, sketching on paper (or a whiteboard) makes the logic dramatically clearer. Trace through examples step by step.

- **Don't skip complexity analysis.** Interviewers almost always ask about time and space complexity. Make sure you can derive it, not just recite it.
- **Review your mistakes.** When you get stuck, don't just look at the answer and move on. Identify *where* your thinking went wrong and *why* the correct approach works.

Looking Ahead: Week 3

In Week 3, you'll build on these foundations with trees (BST operations, tree traversals), dynamic programming basics, and more advanced graph problems. The sliding window and DFS/BFS skills you've developed this week will continue to be essential. Keep up the great work, Sriram — you're making excellent progress!